

Chapter 2

Basic Rendering with OpenGL

Interactive Computer Graphics

Chakrit Watcharopas

Outline

- OpenGL and PyOpenGL
- GLFW
- Primitives in OpenGL
- Face Culling
- How to handle when window size is changed

OpenGL

- ❑ Application Programming Interface (API) used for rendering 2D and 3D scene
- ❑ Primitive Geometry in OpenGL: point, line, triangle, polygon
- ❑ OpenGL is designed to be platform-independent
- ❑ OpenGL takes advantage of hardware acceleration and GPU to improve graphics performance
- ❑ OpenGL renders a scene to framebuffer
- ❑ OpenGL's hardware-accelerated capabilities enable interactive, real-time rendering
- ❑ Microsoft's Direct3D, akin to OpenGL, is a graphics API, an alternative option for developers

What OpenGL won't do?

- ❑ OpenGL does not provide built-in management for framebuffers
 - ❑ developers need to create framebuffer for OpenGL
- ❑ No window management
- ❑ Does not include built-in support for user interaction, such as handling mouse input and keyboard input
- ❑ Render only simple primitives like points, lines, and triangles
 - ❑ developers are responsible for rendering complex model (horse, teapot, bunny)

GLFW (OpenGL Framework)

- ❑ Graphics Library Framework for managing user interaction and window management in OpenGL applications
- ❑ Handles user interaction
- ❑ Manages windows
- ❑ Does not provide GUI Widgets
 - ❑ we will use **ImGui** to create GUI Widgets
- ❑ Facilitates event-driven programming

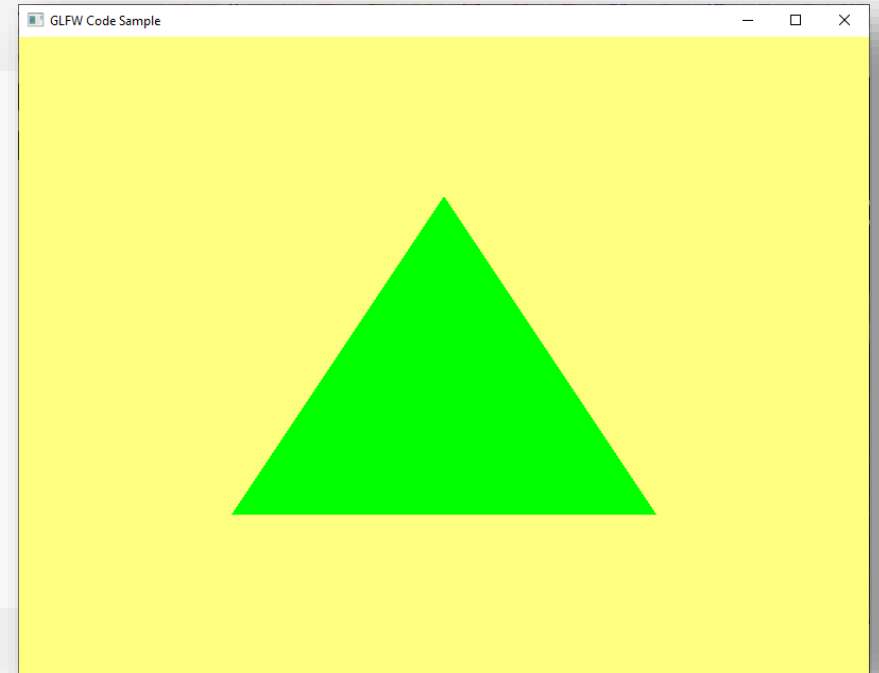
GLFW Code Sample

```
1  if not glfwInit():
2      glfwTerminate()
3      exit()
4
5  glfwWindowHint(GLFW_DOUBLEBUFFER, GL_FALSE)
6  window = glfwCreateWindow(800, 600, "GLFW Code Sample", None, None)
7  glfwMakeContextCurrent(window)
8  glfwSetWindowRefreshCallback(window, refresh)
9
10 while not glfwWindowShouldClose(window):
11     refresh(window)
12     glfwWaitEvents() # or glfwPollEvents()
13 glfwDestroyWindow(window)
14 glfwTerminate()
```

Example of Refresh Callback Function

```
1 def refresh(window):
2     glClearColor(1, 1, 0.5, 0)
3     glClear(GL_COLOR_BUFFER_BIT)
4     glColor3f(0, 1, 0)
5     glBegin(GL_TRIANGLES)
6     glVertex3f(-0.5, -0.5, 0)
7     glVertex3f( 0.5, -0.5, 0)
8     glVertex3f( 0, 0.5, 0)
9     glEnd()
10    glFlush()
```

} OpenGL Code



Explain code each line

- ❑ `glClearColor(1, 1, 0.5, 0)`
 - ❑ sets the color used to clear the color buffer
 - ❑ Red: 1, Green: 1, Blue: 0.5, Alpha (opacity): 0

Value (Red,Green,Blue)	Color	สี
(1, 0, 0)	Red	แดง
(0, 1, 0)	Green	เขียว
(0, 0, 1)	Blue	น้ำเงิน
(1, 1, 0)	Yellow	เหลือง
(1, 0, 1)	Magenta	ม่วง
(0, 1, 1)	Cyan	ฟ้า
(0, 0, 0)	Black	ดำ
(1, 1, 1)	White	ขาว

Explain code each line

- ❑ `glClear(GL_COLOR_BUFFER_BIT)`
 - ❑ clears the color buffer of the currently bound framebuffer with the color specified by `glClearColor`
- ❑ `glColor3f(0, 1, 0)`
 - ❑ sets the current color for next rendering objects
- ❑ `Begin(GL_POLYGON)`
 - ❑ specifies the beginning of a sequence of vertices that define a polygon primitive

Explain code each line

- `glVertex3f(x, y, z)`
 - specifies a vertex coordinate (x, y, z) in 3D space
- `glEnd()`
 - marks the end of a sequence of vertex data defining a geometric primitive
- `glFlush()`
 - ensures all previously issued OpenGL commands are executed by the rendering pipeline

Some OpenGL Command Syntax

- ❑ Begins with the letter "gl"
- ❑ Then followed by command names, e.g. Vertex, Color, etc.
- ❑ Suffixes of the command provide information about arguments passed to the function, e.g.
 - ❑ 3f denotes the function expects three parameters of float
 - ❑ `glVertex3f(1.0, 3.0, 4.0);`
 - ❑ 2i denotes the function expects two parameters of int
 - ❑ `glVertex2i(-1, 5);`
 - ❑ 3fv indicates the function expects a pointer to an array of three floating-point values
 - ❑ Example in C

```
float colorArray[] = {1.0f, 0.0f, 0.0f};
glColor3fv(colorArray);
```
 - ❑ Example in Python (can use tuple or list as an argument)

```
q = [1.5, 2.5, 3.0]
glVertex3fv(q)
```

Command Suffixes and Argument Data Types

Suffix	Data Type	Typical Corresponding C-Language Type	OpenGL Type Definition
b	8-bit integer	signed char	GLbyte
s	16-bit integer	signed short	GLshort
i	32-bit integer	int	GLint, GLsizei
f	32-bit floating-point	float	GLfloat, GLclampf
d	64-bit floating-point	double	GLdouble, GLclampd
ub	8-bit unsigned integer	unsigned char	GLubyte
us	16-bit unsigned integer	unsigned short	GLushort
ui	32-bit unsigned integer	unsigned int	GLuint, GLenum, GLbitfield

Source: OpenGL Programming Guide 8th Edition (2013)

OpenGL is State Machine

- ❑ In OpenGL, "state" specifically refers to various settings and parameters that control the behavior of the rendering pipeline
- ❑ "machine" refers to transitions between different states based on inputs or commands
- ❑ Once a state is set, it remains in effect until explicitly changed by another OpenGL command
- ❑ Example of States
 - ❑ clear color state is set by `glClearColor()`
 - ❑ current color state is set by `glColor*()`
 - ❑ transformation matrix state

How to check OpenGL Version

```
lists = [['Vendor', GL_VENDOR],  
         ['Renderer', GL_RENDERER],  
         ['OpenGL Version', GL_VERSION],  
         ['GLSL Version', GL_SHADING_LANGUAGE_VERSION]]  
  
for x in lists:  
    print('{0}: {1}'.format(  
        x[0], glGetString(x[1]).decode("utf-8")))
```

Geometric Primitives in OpenGL

Geometric Primitives in OpenGL

- ❑ OpenGL can draw simple primitives
 - ❑ point
 - ❑ straight line
 - ❑ triangle and polygon
- ❑ No basic primitives that are curves or curved surfaces
- ❑ We can draw curves by drawing short straight lines in succession



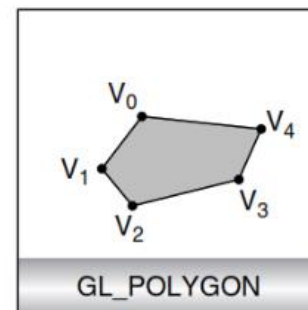
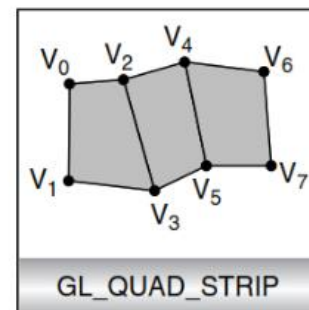
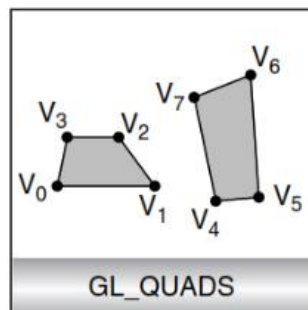
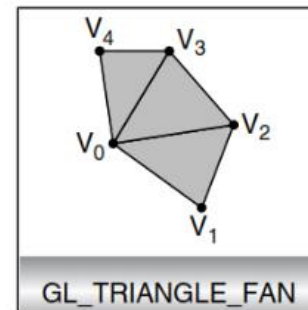
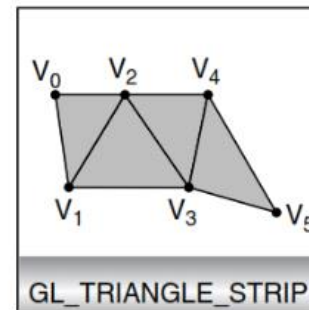
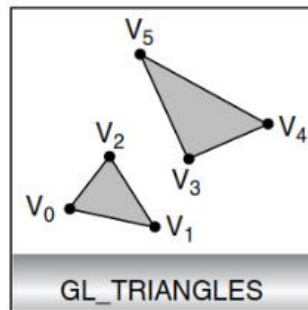
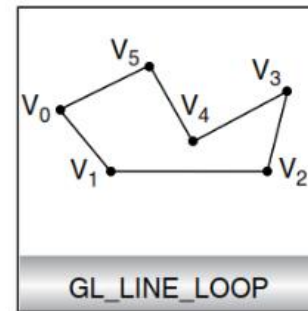
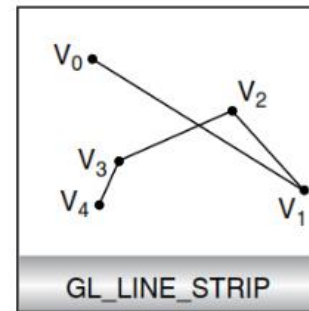
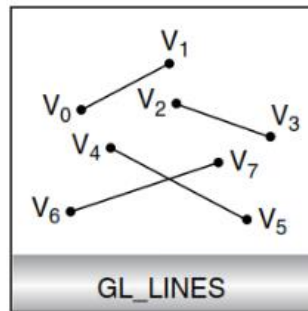
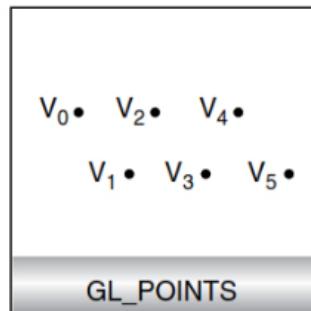
How to Draw Geometric Primitive

- ❑ Start with glBegin(type of primitives)
- ❑ Then use glVertex* to specify the vertices of that primitive
- ❑ End with glEnd()

- ❑ Example of code

```
glBegin(GL_POLYGON)
glVertex3f(-0.5, -0.5, 0.0)
glVertex3f( 0.5, -0.5, 0.0)
glVertex3f( 0.5,  0.5, 0.0)
glVertex3f(-0.5,  0.5, 0.0)
glEnd()
```

Geometric Primitives in OpenGL



glColor

- sets the current color state for rendering subsequent primitives, such as points, lines, or polygons
- Full command syntax
 - `glColor[34][fd][v](colors)`
- Example of `glColor4f(r, g, b, a)`
 - where r, g, b, a represent values of red, green, blue, and alpha (transparency)
 - e.g., `glColor4d(1.0, 1.0, 0.0, 0.4)` sets current color to yellow with 40% opacity
- **Note:** Though we set the alpha value for a color, OpenGL won't utilize it for opacity (or transparency) unless alpha blending is enabled explicitly

Drawing Example

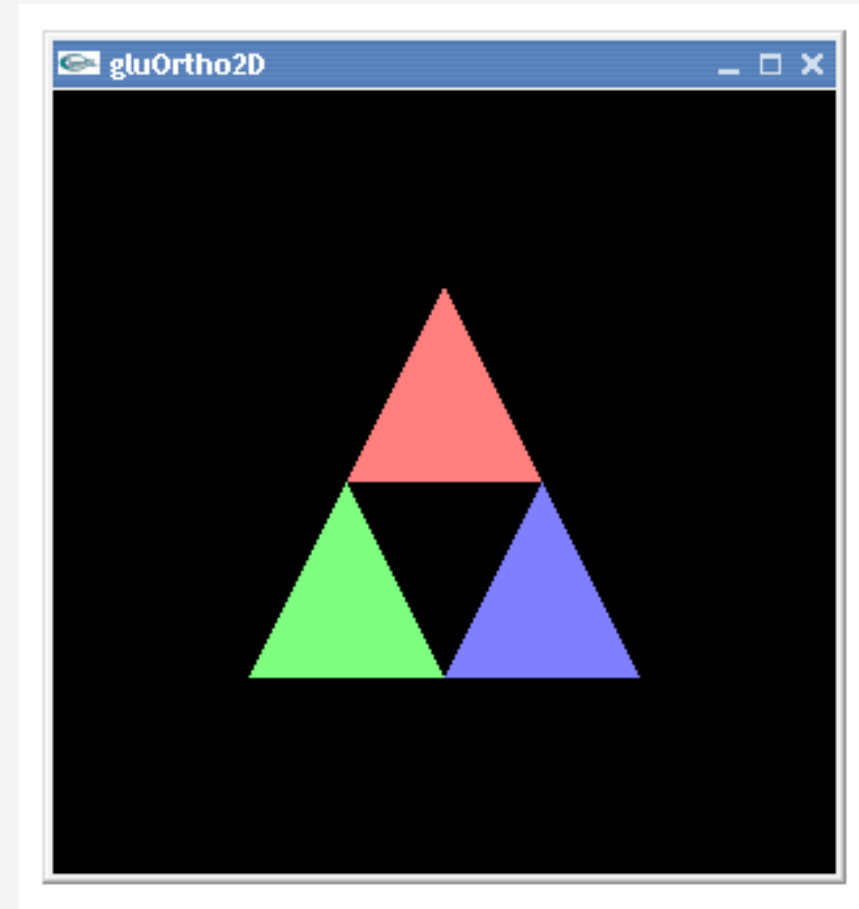
```
glBegin(GL_TRIANGLES)

glColor3f(1.0, 0.5, 0.5) # Red
glVertex3f( 0.0, 0.5, 0.0)
glVertex3f(-0.25, 0.0, 0.0)
glVertex3f( 0.25, 0.0, 0.0)

glColor3f(0.5, 1.0, 0.5) # Green
glVertex3f( -0.25, 0.0, 0.0)
glVertex3f( -0.5, -0.5, 0.0)
glVertex3f( 0.0, -0.5, 0.0)

glColor3f(0.5, 0.5, 1.0) # Blue
glVertex3f( 0.25, 0.0, 0.0)
glVertex3f( 0.0, -0.5, 0.0)
glVertex3f( 0.5, -0.5, 0.0)

glEnd()
```



Drawing Example

```
glBegin(GL_QUADS)

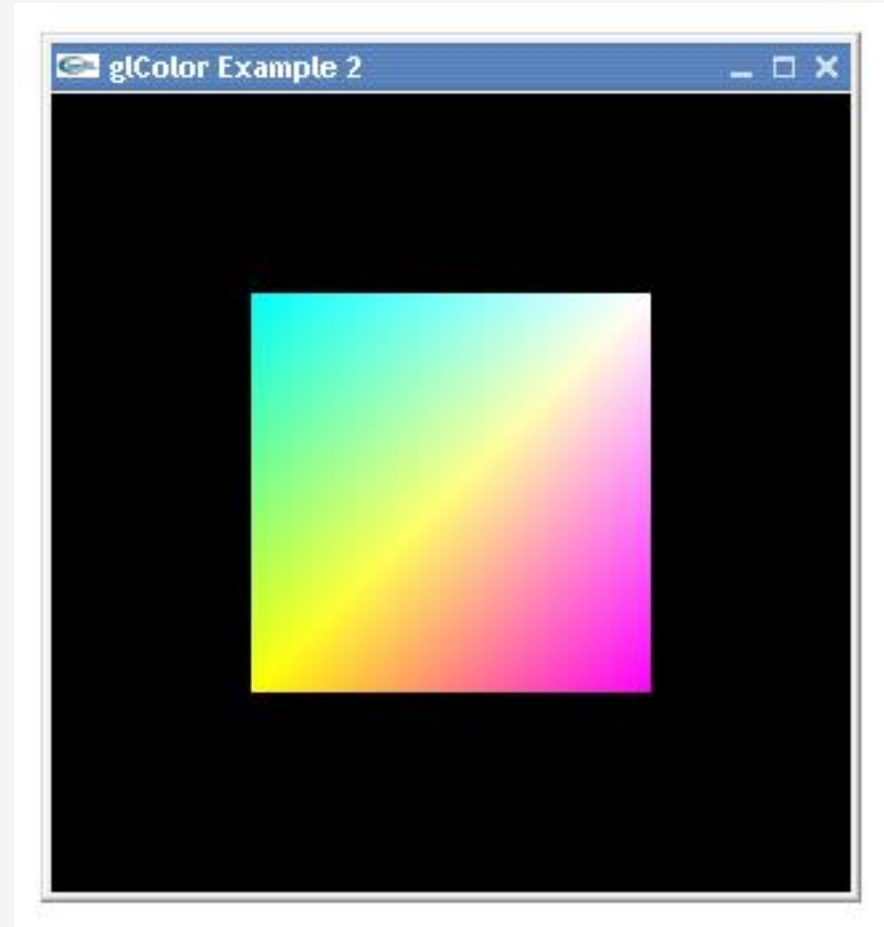
glColor3f( 1.0, 1.0, 0.0)
glVertex3f(-0.5, -0.5, 0.0)

glColor3f( 1.0, 0.0, 1.0)
glVertex3f( 0.5, -0.5, 0.0)

glColor3f( 1.0, 1.0, 1.0)
glVertex3f( 0.5, 0.5, 0.0)

glColor3f( 0.0, 1.0, 1.0)
glVertex3f(-0.5, 0.5, 0.0)

glEnd()
```



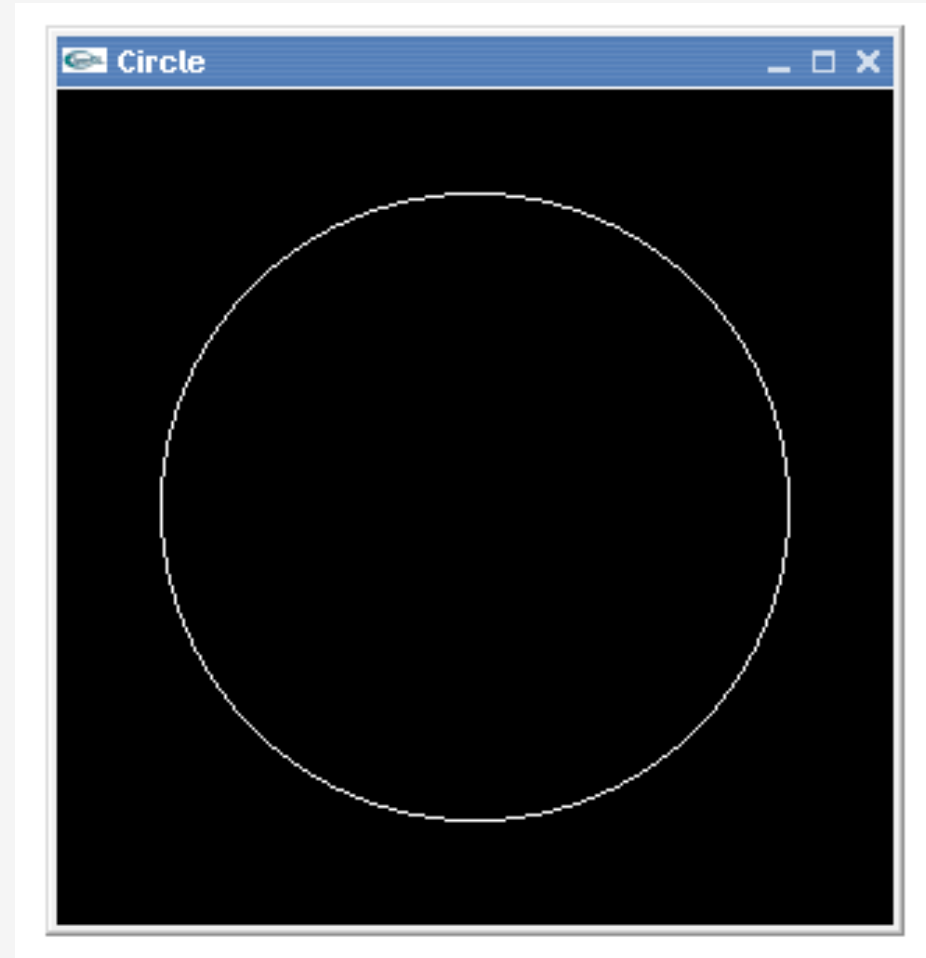
Drawing Example

```
from math import pi, sin, cos

glBegin(GL_LINE_LOOP)

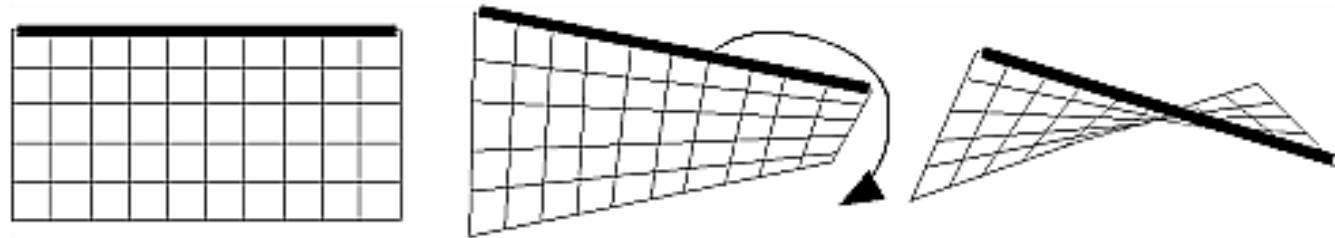
for i in range(360):
    theta = i*2*pi/360
    y = 0.75*sin(theta)
    x = 0.75*cos(theta)
    glVertex2f(x,y)

glEnd()
```



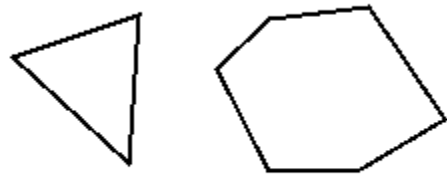
Drawing Polygon in OpenGL

- ❑ OpenGL guarantees it can accurately draw polygons whose vertices are all on the same plane
 - ❑ If not, the result may not be as intended
- ❑ **Observation:** every point that lies on the triangle is on the same plane
 - ❑ But this is not always true for rectangles or other polygons

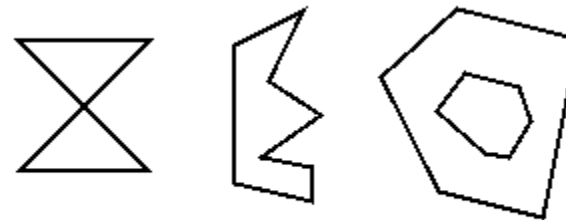


Drawing Polygon in OpenGL

- ❑ Polygons drawn using `glBegin(GL_POLYGON)` must have the following properties:
 - ❑ their edges must not intersect
 - ❑ they must be convex polygons
 - ❑ they must not have holes



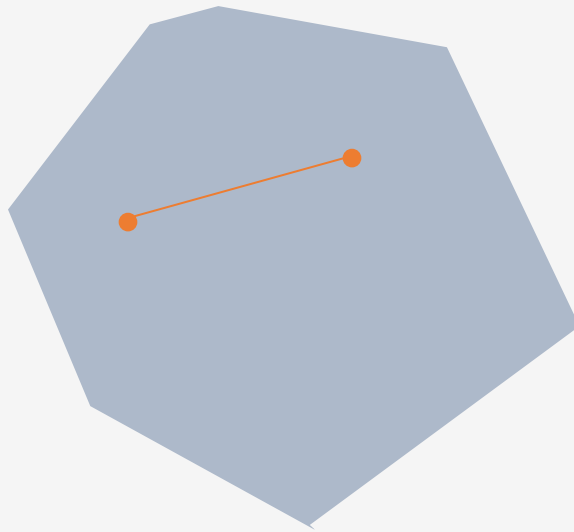
Valid



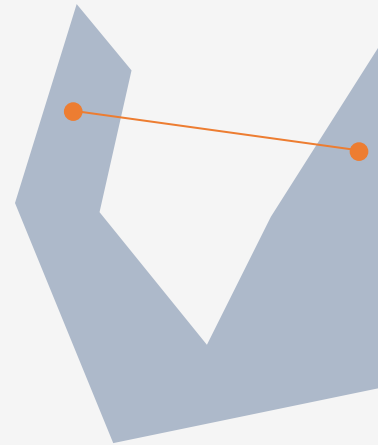
Invalid

Convex Polygon (รูปหลายเหลี่ยมนูน)

- For any two points within a polygon, when a straight line connecting those two points is drawn, that line segment must also lie within the polygon
- **Note:** triangles are always convex



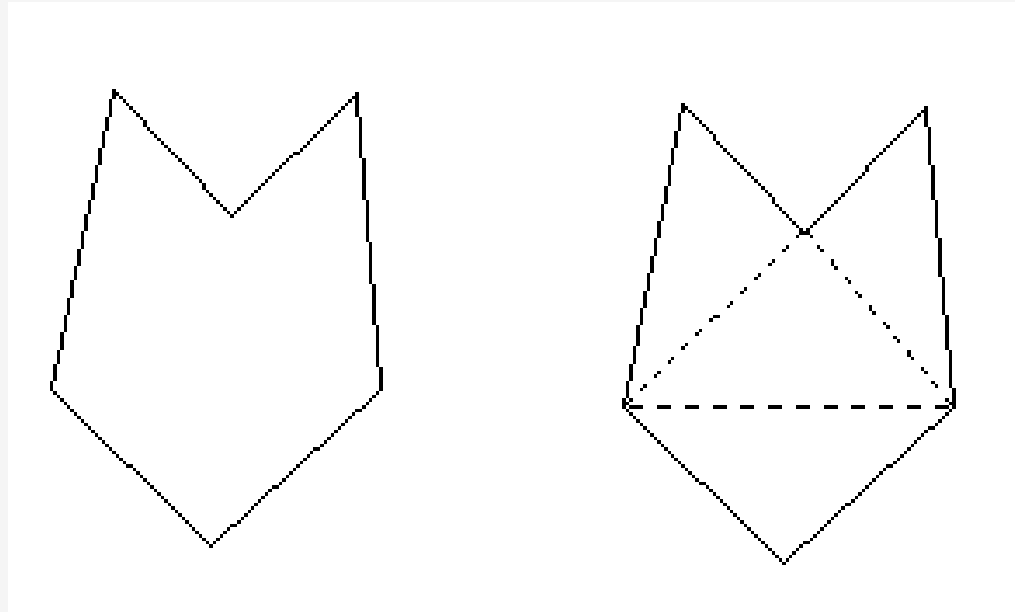
Convex Polygon



Non-Convex or
Concave Polygon

Solution for Drawing Concave Polygon in OpenGL

- Then, how do we draw polygons that are not convex?
- Divide those polygons into multiple triangles



Face Culling

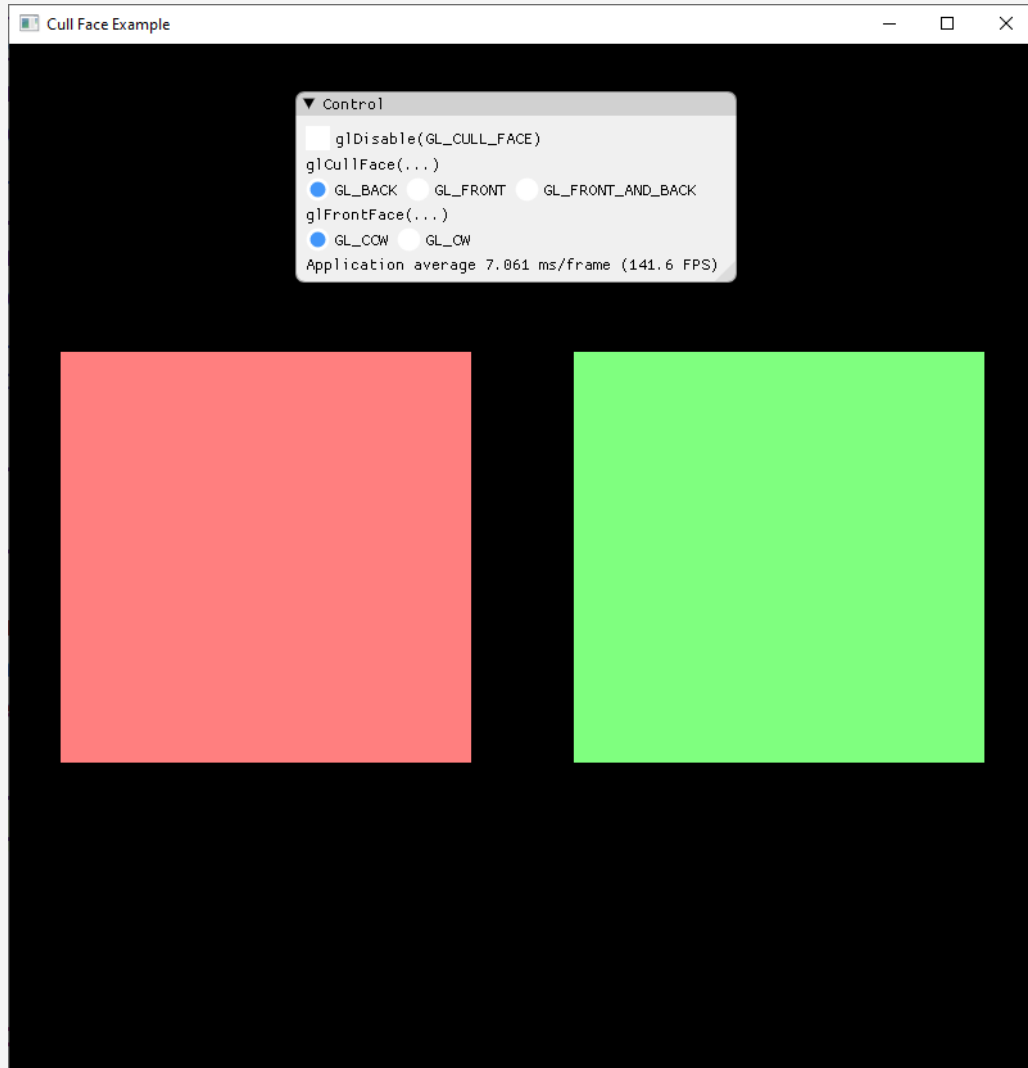
glCullFace

- ❑ `glCullFace(mode)` sets the polygon culling mode
 - ❑ `mode` can take three values
 - ❑ `GL_FRONT` = do not draw the front faces
 - ❑ `GL_BACK` = do not draw the back faces
 - ❑ `GL_FRONT_AND_BACK` = do not draw both side
- ❑ Do not call this command between `glBegin(...)` and `glEnd()`
- ❑ Culling takes effect only when `glEnable(GL_CULL_FACE)` is called beforehand
- ❑ Disable culling (draw all sides) by calling `glDisable(GL_CULL_FACE)`

How to determine Front Face

- Normally, for polygons where we see the vertices arranged in a clockwise direction, they are considered "front-facing"
- Conversely, if we see the vertices arranged in a counterclockwise direction, they are considered "back-facing"
- However, we can change this using `glFrontFace(...)`
 - `glFrontFace(GL_CCW)` specifies counterclockwise as the front face
 - `glFrontFace(GL_CW)` specifies clockwise as the front face

Demo



```
glBegin(GL_QUADS)
```

```
glColor3f(1.0, 0.5, 0.5)
glVertex2f(-0.9, -0.4)
glVertex2f(-0.1, -0.4)
glVertex2f(-0.1, 0.4)
glVertex2f(-0.9, 0.4)
```

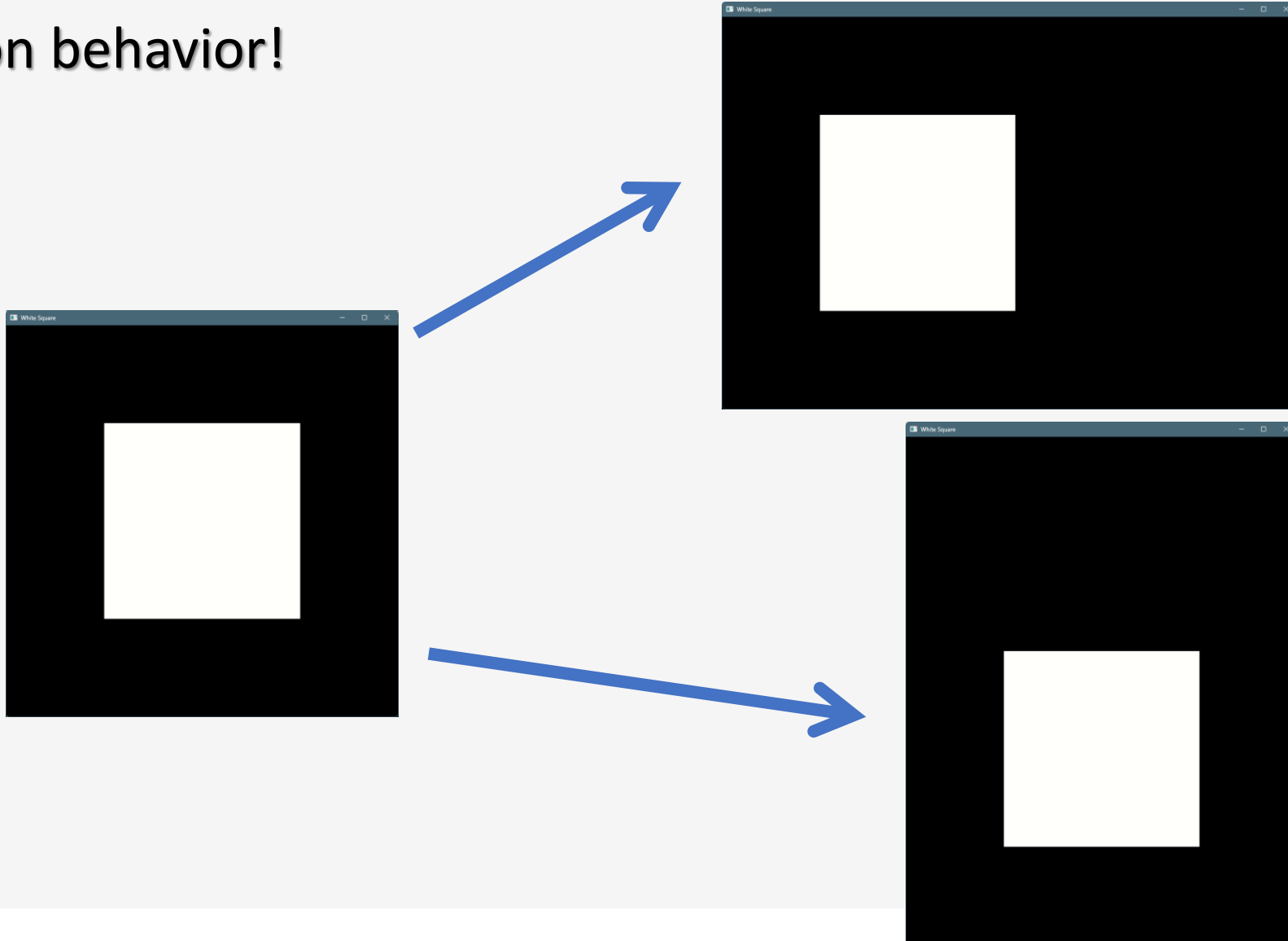
```
glColor3f(0.5, 1.0, 0.5)
glVertex2f(0.1, -0.4)
glVertex2f(0.1, 0.4)
glVertex2f(0.9, 0.4)
glVertex2f(0.9, -0.4)
```

```
glEnd()
```

How to handle when a window is resized

When a window is resized

- Common behavior!



Default Vertex Coordinate Range on Window

- ❑ In OpenGL, when drawing vertices to the window, the default coordinate range often follows conventions of normalized device coordinates (NDC)
- ❑ In NDC, the coordinate range typically spans from -1.0 to 1.0 along each axis:
 - ❑ X-axis: The left edge of the window is usually mapped to -1.0, while the right edge is mapped to 1.0
 - ❑ Y-axis: The bottom edge of the window is often mapped to -1.0, and the top edge is mapped to 1.0
 - ❑ Z-axis: The depth range usually extends from -1.0 (near plane) to 1.0 (far plane)
- ❑ The default viewport mapping in OpenGL refers to how the NDC space is mapped to the window or framebuffer
- ❑ By default, OpenGL sets up a viewport that matches the dimensions of the window or framebuffer
- ❑ The point (-1.0, -1.0) in NDC maps to the bottom-left corner of the window.
- ❑ The point (1.0, 1.0) in NDC maps to the top-right corner of the window.

glfwSetWindowSizeCallback

- ❑ `glfwSetWindowSizeCallback(window, resize_callback)`
 - ❑ A resize callback function provided to `glfwSetWindowSizeCallback` must be defined as follows:
def <function_name>(window, width, height):
 - ❑ The function takes a GLFW window object and two integers as arguments
 - ❑ This function will be called every time the window is resized
 - ❑ The two integer arguments represent the new width and height of the window in pixels

Example code for glfwSetWindowSizeCallback

```
def resize(window, w, h):
    glViewport(0, 0, w, h)

def refresh(window):
    glClearColor(0, 0, 0, 0)
    glClear(GL_COLOR_BUFFER_BIT)
    glBegin(GL_POLYGON)
    glColor3f(1, 1, 1)
    glVertex3f(-0.5, -0.5, 0)
    glVertex3f( 0.5, -0.5, 0)
    glVertex3f( 0.5,  0.5, 0)
    glVertex3f(-0.5,  0.5, 0)
    glEnd()
    glFlush()
```

```
if not glfwInit():
    glfwTerminate()
    exit()

glfwWindowHint(GLFW_DOUBLEBUFFER, GL_FALSE)
window = glfwCreateWindow(800, 800, "Resize", None, None)
glfwMakeContextCurrent(window)
glfwSetWindowRefreshCallback(window, refresh)
glfwSetWindowSizeCallback(window, resize)

while not glfwWindowShouldClose(window):
    refresh(window)
    glfwPollEvents()

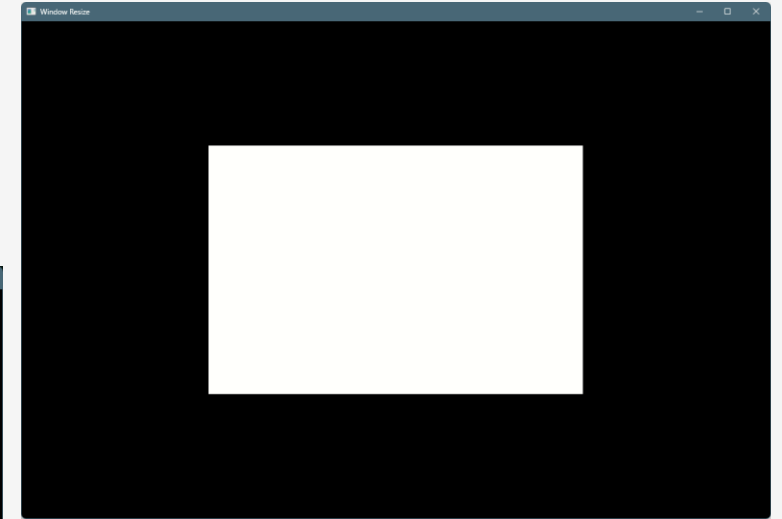
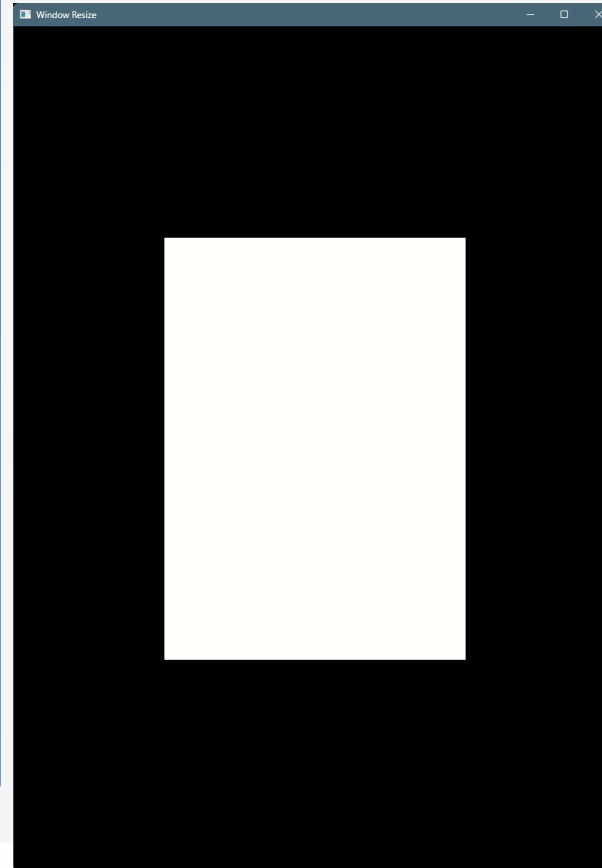
glfwTerminate()
```

glViewport

- ❑ `glViewport(x, y, w, h)` sets the area within the window that OpenGL will use to draw
- ❑ The coordinates `(x, y)` specify the position of the bottom-left corner of the area
- ❑ The `w` and `h` specify the width and height of the area
- ❑ In the example, we use `glViewport(0, 0, w, h)`, which means using the entire window area to display

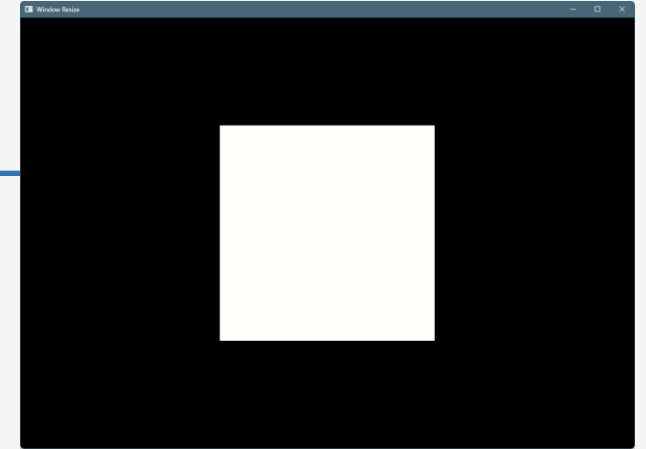
Results when window is resized

```
def resize(window, w, h):  
    glViewport(0, 0, w, h)  
  
def refresh(window):  
    glClearColor(0, 0, 0, 0)  
    glClear(GL_COLOR_BUFFER_BIT)  
    glBegin(GL_POLYGON)  
    glColor3f(1, 1, 1)  
    glVertex3f(-0.5, -0.5, 0)  
    glVertex3f( 0.5, -0.5, 0)  
    glVertex3f( 0.5,  0.5, 0)  
    glVertex3f(-0.5,  0.5, 0)  
    glEnd()  
    glFlush()
```

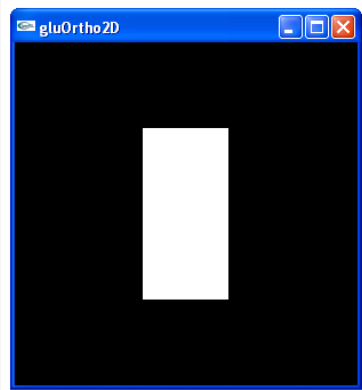


If we want polygon to remain squared

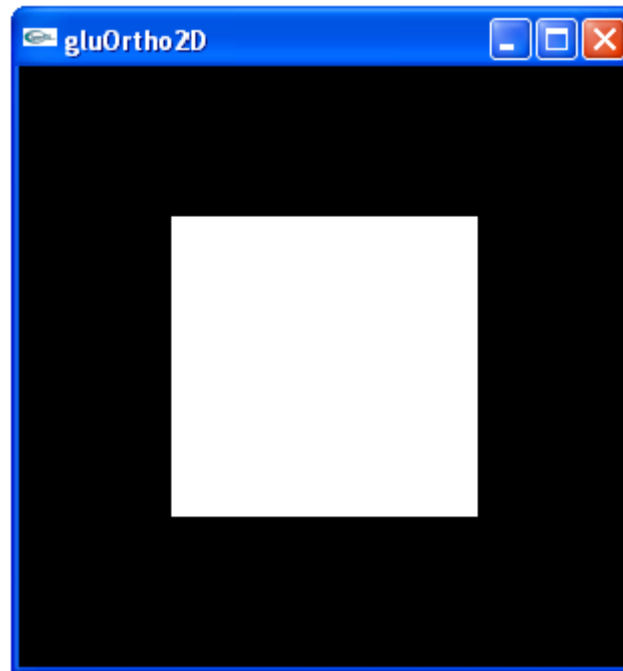
- ❑ We need to adjust range of vertex coordinate accordingly, when window size is changed
- ❑ `gluOrtho2D` sets coordinate range along x-axis and y-axis
- ❑ `gluOrtho2D(left, right, bottom, top)` takes parameters to transform the projection matrix into an orthographic projection (Discuss in Week 3)
- ❑ Think of it as specifying corners of the area where to draw shapes
 - ❑ bottom-left corner of the window is (left, bottom)
 - ❑ bottom-right corner of the window is (right, bottom)
 - ❑ top-left corner of the window is (left, top)
 - ❑ top-right corner of the window is (right, top)



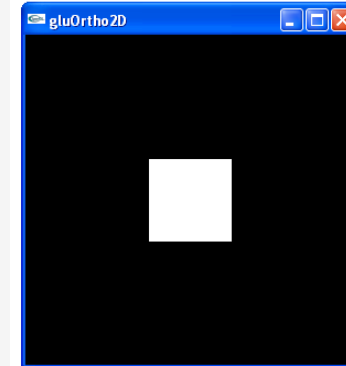
Example uses of gluOrtho2D



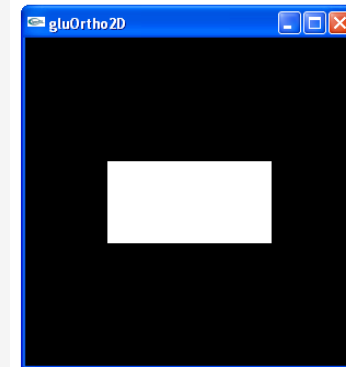
$\text{gluOrtho2D}(-2,2,-1,1)$



$\text{gluOrtho2D}(-1,1,-1,1)$



$\text{gluOrtho2D}(-2,2,-2,2)$



$\text{gluOrtho2D}(-1,1,-2,2)$

Put together and adjust Resize() callback

- ❑ We call `glMatrixMode(GL_PROJECTION)` and `glLoadIdentity()` before calling `gluOrtho2D(...)` to set up the coordinate system
- ❑ Both functions are related to setting the projection transform, which we will discuss in week 3
- ❑ Remember these two functions must always be called before calling `gluOrtho2D`
- ❑ Using `gluOrtho2D`, we need to import `OpenGL.GLU`

```
def resize(window, w, h):  
    glViewport(0, 0, w, h)  
    glMatrixMode(GL_PROJECTION)  
    glLoadIdentity()  
    # choose a shortest side  
    #           to have range of [-1, 1]  
    if w < h:  
        gluOrtho2D(-1, 1, -h/w, h/w)  
    else:  
        gluOrtho2D(-w/h, w/h, -1, 1)
```

References

- Dave Shreiner, Bill The Khronos OpenGL ARB Working Group, et al. OpenGL Programming Guide: The Official Guide To Learning OpenGL, Versions 3.0 and 3.1. Pearson Education, 2009.
- Dave Shreiner and Edward Angel. Interactive Computer Graphics: A top-down approach with OpenGL, 2nd Edition, 2009.
- Fletcher, M. *PyOpenGL (The Python OpenGL® Binding)*. Available at: <http://pyopengl.sourceforge.net>, accessed Oct 14, 2011.
- GLFW Community. *GLFW Documentation*. <https://www.glfw.org/documentation>, accessed May 14, 2024.